Blit Order Guide: Keeping Every Redraw Ahead of the Beam

A companion guide to Building an AMIGA Game Engine

Richard Shannon

AmigaGameEngine v1.0

Contents

1. Terms 1

2. One Frame, In Order 2
- Phase 1: priority work (bar yellow) . . . 2
- Phase 2: the rest (bar green) . . . 2
- Which phase the player belongs to . . . 3

3. Layering This Order Produces 3

4. The Rules 3
- Rule 1: Draw top of the screen first . . . 4
- Rule 2: Priority work is the top tier, and nothing else . . . 4
- Rule 3: The player's core runs before any entity that could overlap it . . . 4
- Rule 4: Keep the player's core fast; slow work goes in the extras . . . 4
- Rule 5: An object's water tint follows its blit immediately . . . 4
- Rule 6: Anything drawn after the entities that can cover them must be followed by redrawing them . 4
- Rule 7: Everything the player's erase can wipe must be repaired in the same pass . . . 4
- Rule 8: A redraw outside the draw pass must wait for the beam . . . 4
- Rule 9: Address the copy that is on screen . . . 5
- Rule 10: Never repaint the visible rows during the frame . . . 5
- Rule 11: The camera moves after drawing . . . 5

5. Paths Outside the Normal Frame 5

6. Adding a New Drawn Object: Checklist 5

7. Verifying 6

AmigaGameEngine draws into a **single displayed buffer** (`DisplayScreen`). There is no back buffer to flip to, so every object the engine redraws is visible the moment its pixels change. Each redraw is an *erase, then draw* (copy clean background from `NonDisplayScreen`, then cookie-cut the new image). If the electron beam reaches the object's scanlines between those two steps, the monitor shows that frame half drawn: a missing lower half, a clear rectangle, or a sprite without its water tint.

This guide records the order the engine draws in, and the rules that keep that order safe. **Read it before adding, moving or reordering any blit.** Each rule below exists because breaking it produced a visible glitch.

Source of truth: `GameRunFrame` in `engine/gameplay/gamestatus.asm`. If this guide and the code ever disagree, the code wins and this guide must be updated in the same commit. `BLIT_ORDER_GUIDE.pdf` is built from this file by `docs/build_docs_pdf.ps1`: rebuild it whenever you edit this guide.

1. Terms

Term	Meaning
Screen Y	World Y minus `TilemapCameraY`: 0 is the top line of the display window.
Raster line	Screen Y + `SpriteYOffset` (the display window's first line: `$2C` PAL, `$1C` NTSC). This is what `VPOSR`/`VHPOSR` report.
Top tier	Screen Y < `ENTITY_TOP_TIER_Y` (48, the top three tile rows). The beam reaches these lines first.
Phase 1 / priority work	Everything that must be drawn before the beam reaches the top tier. The raster profiler bar (F5 then F3) is **yellow** while it runs.
Phase 2	Everything else. `SetRasterLogicColor` turns the bar **green** the moment Phase 1's work is finished, wherever the beam is.
Buffer line	A line of the 512-line ring buffer: 16 primary tile rows plus a 16-row mirror (Chapter 27).

Reading the bar: if no yellow shows inside the display window, all priority work finished before the beam reached the first visible line. Yellow that runs into the picture is a bug.

2. One Frame, In Order

`MainLoop` waits for `VBlankTick` (level 3, line 0) to set `FramePending`, sets `COLOR00` yellow when the profiler is on (red if a frame has overrun), then runs the state machine. In gameplay that is `GameRun`, whose guards run first (level-done test, ESC fade, pause, slow mode, F9 / VHS rewind, see §5), then `GameRunFrame`.

At line 0 the Copper restarts from `COP1LC` and loads the bitplane pointers (`cpPlanes`), palette and sprite pointers that the *previous* frame's logic patched. The camera position for the whole frame is therefore fixed before anything is drawn.

Phase 1: priority work (bar yellow)

#	Step	Routine	What it draws
1	Player core, **only if the player is in the priority phase**	`GameRunDrawPlayerCore`	`TilemapErasePlayer` (one copy blit of the old 32/48 × 24 footprint), `TilemapDrawPlayerCore` (cookie-cut BOB), `TilemapSubmergePlayer` (water tint, one compare when dry)
2	Top-tier entities, top of the screen first	`GameDrawPriorityHook` with `d0 = -32768`, `d1 = ENTITY_TOP_TIER_Y` (Duck Rescue: `TilemapUpdateEnemiesInRange`)	For each enemy in `EnemyDrawOrder` in the slice: move, erase old footprint, re-blit entities it touched, redraw the player if its erase cut the player, cookie-cut the enemy, stun overlay, water tint
3	Player extras, **only if the player is in the priority phase**	`GameRunDrawPlayerExtras`	`TilemapStampOverPlayer` (foreground stamp, ladder-scenery stamp), then `GameRestoreOverlapsHook` (Duck Rescue: `TilemapRestorePlayerOverlaps`, then `TilemapRedrawEnemiesOverPlayer`)
4	Bar to green	`SetRasterLogicColor`	—

Phase 2: the rest (bar green)

#	Step	Routine	What it draws
5	Player core, **only if the player is not in the priority phase**	`GameRunDrawPlayerCore`	as step 1
6	Remaining entities, top first	`GameDrawPriorityHook` with `d0 = ENTITY_TOP_TIER_Y`, `d1 = 32767`	as step 2
7	Player extras, **only if not in the priority phase**	`GameRunDrawPlayerExtras`	as step 3
8	Remaining game blits	`GameDrawHook` (Duck Rescue: `PlayerPuffRun`, the landing puff)	restores the puff tile, redraws the player (with stamps and tint), draws the puff frame

#	Step	Routine	What it draws
9	Input and game logic	`UpdateControls`, `GameLogicHook`	**Pushed crates** are erased and redrawn here by `ActionPush` (with the player redrawn and tinted), after `WaitBeamClearOfLines` (§4, rule 8)
10	Camera	`TilemapUpdateCamera`	Repoints `cpPlanes` (takes effect next frame). Streams newly revealed rows into the ring buffer (`TilemapStreamTileRow`); these rows are outside the visible window. A jump of 16+ rows rebuilds the whole buffer and re-applies the water.
11	Water	`TilemapUpdateWater`	Moves the waterline: rewrites the wave scanlines in both buffers and both halves
12	Post-logic	`GamePostLogicHook`	Oxygen, HUD sprite pointers (Copper list words, no blits)
13	Tile animation	`AnimateEnemies`	Animated tile frames
14	Falling crates, landing puffs	`GameActorPhaseHook` (`ActionFallActors`)	Each falling crate: `WaitBeamClearOfLines`, `ClearActor`, restore its two background tiles, `DrawActor` one step lower; landing smoke frames
15	Settled crates	`TilemapDrawPushBlocks`	Only crates marked dirty
16	Static entities	`GameDrawEntitiesHook`	`TilemapDrawPickups` (only undrawn or collected pickups), `TilemapUpdateFriends` (friends redraw when their animation steps)
17	Dirty tiles	`FlushDirtyTiles`	Settled actors / frozen player at cells marked dirty
18	Debug overlay	`TilemapDrawDebugOverlay` (slow mode only)	Telemetry text

When the frame finishes, `MainLoop` sets `COLOR00` black and clears `FrameBusy`. If no VBlank fired during the frame, it also clears `VBlankOverrunCount`. It then waits for the next VBlank.

Which phase the player belongs to

`PlayerDrawKeyY` returns the higher (smaller screen Y) of the player's old footprint (`Player_PrevY`, if drawn last frame) and its new one, moved up a further 15 lines. An enemy is 16 lines tall, so an enemy whose top is at or below that key could overlap the player's rows. If the key is < `ENTITY_TOP_TIER_Y`, a top-tier enemy could overlap the player, so the player's core must run in Phase 1, before those enemies. Otherwise it runs at the start of Phase 2.

3. Layering This Order Produces

From back to front, wherever objects overlap:

1. Background tiles (`NonDisplayScreen`, copied in by every erase)
2. Ladder-side scenery (stamped over the player while climbing)
3. **Player**
4. Crates, pickups, friends caught in the player's erase (`TilemapRestorePlayerOverlaps`)
5. **Enemies** (drawn after the player core; put back in front of the player's stamps by `TilemapRedrawEnemiesOverPlayer`)
6. Foreground tiles (stamped over the player by `TilemapStampOverPlayer`, and over every enemy, crate and pickup by their own draw routines)
7. Water tint (applied straight after each object's blit, by the object's own routine)

Hardware sprites (HUD, air bubble) are not blitted and cannot tear this way: Denise overlays them on the playfield (`BPLCON2 = $0024` puts sprites 0–3 in front of it), and their pointers are Copper words patched in the logic for the next frame.

4. The Rules

Each rule names the glitch it prevents.

Rule 1: Draw top of the screen first

Entities are walked in `EnemyDrawOrder` (pointers sorted by `Enemy_Y`, kept in order by `SortEnemyDrawOrder`). An object near the top has the least time before the beam reaches it, so it goes first.

> *Breaks as:* a top-row enemy cut off or blank for a frame.

Rule 2: Priority work is the top tier, and nothing else

Phase 1 holds only what is in the top three rows: the top-tier entities and, when its key is there, the player. Everything lower waits for Phase 2. Do not move work into Phase 1 "to be safe": it delays the top-tier objects that actually need it.

> *Breaks as:* yellow running into the display, and top-row objects torn.

Rule 3: The player's core runs before any entity that could overlap it

The player's erase copies clean background over a 32- or 48-pixel-wide rectangle. Anything already drawn there is wiped. Running the core first means the erase has already happened when the entities are drawn, so they are drawn on top of the player and cannot be wiped by it.

> *Breaks as:* a clear rectangle cutting into an enemy beside the player.

Rule 4: Keep the player's core fast; slow work goes in the extras

The core is two blits plus the tint check. The stamps and overlap repair are slower, so they run after the entities, never between the player's core and an entity.

> *Breaks as:* an enemy beside the player drawn too late for the beam (a cut-off lower half), when the whole player draw ran first.

Rule 5: An object's water tint follows its blit immediately

`TilemapSubmergePlayer` is part of the player core. Enemies, friends, crates and pickups tint themselves inside their draw routines. Never defer a tint to later in the frame.

> *Breaks as:* the player flickering dry for a frame while underwater.

Rule 6: Anything drawn after the entities that can cover them must be followed by redrawing them

The ladder-scenery stamp runs after the entities (rule 4) and re-blits scenery tiles beside the ladder. `TilemapRedrawEnemiesOverPlayer` then redraws, at the same position, every enemy overlapping the stamped area. If you add another stamp or redraw to the extras, check whether it can cover an entity, and extend that routine's area if so.

> *Breaks as:* an enemy vanishing for a few frames one tile beside the player.

Rule 7: Everything the player's erase can wipe must be repaired in the same pass

`TilemapRestorePlayerOverlaps` redraws the settled, pushed **and falling** crates, pickups and friends whose tiles the erase touched. A new kind of object that can sit beside the player needs a case there.

> *Breaks as:* a crate falling beside the player with part of it missing.

Rule 8: A redraw outside the draw pass must wait for the beam

Some objects are redrawn during the game logic, after the draw pass: a crate being pushed (`ActionPush`) and a crate falling (`ActionFallActors`). Before erasing, they call `WaitBeamClearOfLines` (`engine/system/tools.asm`) with the raster lines they are about to rewrite. It returns at once unless the beam is within `BEAM_CLEAR_MARGIN` (16) lines above the band or on it, and then waits until the beam has passed. Any new mid-frame redraw needs the same call, with a band that covers everything it erases (for example, both background tiles a falling crate

restores).

> *Breaks as:* a crate near the top of the view half drawn while it moves.

Rule 9: Address the copy that is on screen

The ring buffer holds every tile row twice: buffer lines 0–255 and a mirror at 256–511, so the visible window never wraps (Chapter 27). An object drawn this frame lives at the buffer line its draw used: `(TilemapCameraY & 255) + (Y - TilemapCameraY)` (the player records the camera it used in `PrevPlayerCameraY`). Anything that touches that object afterwards (erase, tint, overlay) must use the same line. Code that works on tiles rather than drawn objects must write **both** halves (`TilemapBlitSingleTile`, `TilemapApplyWaveScanline`, `TilemapErase16x16Actor` do). Never address a single copy with `Y & 255`.

> *Breaks as:* an effect that works on short levels but not near the bottom of the view on a tall one (Level 6): the player not tinted underwater.

Rule 10: Never repaint the visible rows during the frame

Row streaming only writes rows outside the visible window. A full ring rebuild (respawn, a long undo) repaints everything, so after it the caller must restore what the rebuild does not draw: the water (`TilemapRenderWaterInBuffer`) and the static objects (crates marked dirty, pickups marked undrawn). See `TilemapCentreCamera` and `PlayerRespawn`.

> *Breaks as:* a respawn or undo showing stale rows, missing water or missing crates.

Rule 11: The camera moves after drawing

`TilemapUpdateCamera` runs in the logic, after the draw pass, and its bitplane pointers take effect at the next frame's line 0. Every blit in a frame therefore uses one camera position. Do not move the camera before or during the draw pass.

5. Paths Outside the Normal Frame

These replace or interrupt `GameRunFrame`. Each hides or handles its redraw differently.

Path	Where	How it avoids tearing
Level reveal	`LevelTransitionRun .normal_reveal` (`levelutils.asm`)	Bitplane DMA is switched off while the level is built; the palette fades up from black
ESC fade	`GameRunQuitFade`	Palette-only; actors frozen
Pause	`GameRunPauseKey`, `pause_system.asm`	Banner drawn once; actors frozen
F9 undo + VHS rewind	`GameRunRewind`, `UndoMove` (`undo.asm`), `vhs_rewind.asm`	Runs at the start of the frame (VBlank). `UndoMove` erases all entities (`GameUndoEraseHook`), restores state, snaps the camera (full rebuild + water if it jumped 16+ rows), redraws crates, the player and entities (`GameUndoRedrawHook`)
Respawn	`PlayerRespawn` (`player_water.asm`), from the logic	`TilemapCentreCamera` rebuilds the whole view and the water, then crates and pickups are forced to redraw. A one-off multi-frame redraw on death
Landing puff	`PlayerPuffRun` in `GameDrawHook`	Restores the puff tile, redraws the player with stamps and tint, then the puff frame

6. Adding a New Drawn Object: Checklist

1. **Does it move or animate every frame?** Draw it in the top-down pass: add it to the sorted walk behind `GameDrawPriorityHook` (or a parallel sorted list walked in the same slices), not in the logic.
2. **Can it be redrawn outside the draw pass?** Then call `WaitBeamClearOfLines` before erasing, with a band covering everything it erases (rule 8).

3. **Can it sit beside the player?** Add it to `TilemapRestorePlayerOverlaps` (rule 7). If it should appear in front of the player's stamps, add it to `TilemapRedrawEnemiesOverPlayer` or an equivalent (rule 6).
4. **Can it be underwater?** Apply its tint inside its own draw routine, straight after its blit, at the address it drew to (rules 5 and 9).
5. **Does it address the buffer?** Use the draw-time buffer line, or write both halves (rule 9).
6. **Test on Level 6** (the 43-row level) with the raster profiler on (F5, then F3), with the object near the top of the view, beside the player, and near the bottom of the view.

7. Verifying

- **Raster bar:** F5 then F3. No yellow inside the display window. Green ending well above the bottom of the screen. Red replaces both colours after a frame overruns, and they return as soon as a frame finishes on time.
- **Slow mode:** S, then tap A to step a frame at a time; tearing that lasts one frame is easy to miss at full speed.
- **Places that have caught bugs before:** Level 6, top-right ladder with an enemy alongside; pushing and dropping the crate near the top of the view; climbing the ladder down into the water near the bottom of the view; dying and respawning far from the safe platform.